

DocProcessor

将文档转化为可验证的功能地图，用于 **QA** 自动化。

概要

DocProcessor 是一个独立、完全解耦的 Go 模块，负责加载项目文档、构建结构化功能地图并跟踪验证覆盖率。它设计用于与 LLM 智能体协同工作，实现智能特征提取，同时也内置启发式提取功能，支持离线使用。

简介

一个与项目无关的 Go 文档处理模块，用于解析文档并提取功能地图。它将文档转化为结构化功能地图，并跟踪已验证的功能——通过 LLM 智能体实现智能提取，或在离线状态下使用启发式算法——为 QA 自动化提供“反虚假承诺”的保证，确保结果始终与实际一致。

详细描述

每个软件团队都面临同一个隐秘的谎言：文档描述的功能与测试覆盖的内容相去甚远，无人能确信两者是否指向同一个产品。DocProcessor 的存在就是为了让这一差距变得可见且可衡量。基于项目文档，它构建出一份结构化的功能地图——即产品宣称实现的所有功能的枚举式、机器可读模型——并跟踪其验证覆盖率，使“这个文档中描述的功能是否已得到实际验证”不再是走廊争论的话题，而是一个可查询的答案。它刻意采用双模式设计：在智能体可用时，使用 LLM 智能体进行语义化特征提取；在离线环境下，则回退至启发式解析器，确保不依赖任何模型，无论是在隔离的 CI 任务中，还是在开发者飞行模式下的笔记本电脑上，均能一致运行。

在架构上，它是一个独立、与项目无关、完全解耦的 Go 模块 (CONST-051(B))：不内置任何项目特定值，由使用方以同等代码库子模块的形式集成，因此任何项目均可采用，无需继承他人的假设。同时，它 also 以身作则，遵循自身强制执行的标准——其所有声明均受“反虚假承诺”约定 (CONST-035) 和全自动化覆盖规则 (CONST-048) 约束，这意味着其 README 中宣称的每一项功能，都由自动化测试或挑战脚本验证，确保真正可用于最终用户，而非仅仅“退出码为零”；面向用户的字符串则通过 CONST-046 国际化翻译接口处理。这一切的核心在于形成闭环：

DocProcessor 是 QA 流程的输入端，由 HelixQA 完成闭环——它从文档中提取功能地图，HelixQA 则通过捕获的运行时证据验证每一项功能，从而迫使文档、测试与实际交付的行为趋于一致，而非在版本迭代中悄然偏离。

为何开发

文档与测试渐行渐远：文档承诺的功能无测试验证，QA 无法轻易判断何为“完整”。DocProcessor 将文档转化为机器可读的功能地图，使验证覆盖率能够以实际承诺的内容为基准进行衡量。

内容

为何堪称变革性突破

它将软件交付中最模糊的问题——“我们交付的产品是否与当初承诺的一致？”——转化为可自动化、持续验证的流程。且这一转化无需依赖硬性的AI条件：无论是否有模型可用，均可通过LLM提取（模型支持时）或启发式算法（模型缺失时）实现，确保从离线运行器到全智能管道的每个环境中，验证结果始终如一。

创新之处

- 文档-功能映射提取，并同步跟踪验证覆盖率。
- 双重提取机制：支持LLM智能代理驱动或启发式/离线模式。
- 项目无关、零配置解耦（CONST-051(B)）。
- 防虚假验证：README中的所有声明均由测试/验证挑战（Challenge）背书（CONST-035/048）。

挑战与解决方案

- 模型可选运行：通过启发式提取器回退机制解决，确保模块在离线环境下正常工作。
- 文档与现实保持一致：通过结构化功能映射+验证覆盖率跟踪解决，并纳入质量保证闭环。
- 复用性：通过严格解耦及同代码库子模块调用实现。
- 自身声明的可信度：通过针对每项宣称功能的防虚假测试/验证挑战解决。

技术栈（选型依据与实现方式）

- **Go (1.25+版本)** —— 模块核心，基于Apache-2.0协议开源。
- **LLM智能代理** —— 语义化功能提取（可选）。
- **启发式解析器** —— 离线功能提取的备用方案。
- **国际化翻译组件 (pkg/i18n)** —— 支持CONST-046本地化字符串。

- 验证挑战框架 —— 对模块自身声明进行防虚假验证。